UNIT-1 (INTRODUCTION TO ALGORITHMS AND ANALYSIS)

FUNDAMENTALS OF ALGORITHM ANALYSIS

LECTURE NO:-1

Algorithm

A process or set of rules to be followed in calculations or other problem-solving operations, especially by a computer.

Algorithm Analysis

Algorithm analysis is an important part of computational complexity theory, which provides theoretical estimation for the required resources of an algorithm to solve a specific computational problem. Analysis of algorithms is the determination of the amount of time and space resources required to execute it.

Importance of Analysis of Algorithms

1) To predict the behavior of an algorithm without implementing it on a specific computer.
2) It is much more convenient to have simple measures for the efficiency of an algorithm than to implement the algorithm and test the efficiency every time a certain parameter in the underlying computer system changes.
3) It is impossible to predict the exact behaviour of an algorithm. There are too many influencing factors.
4) The analysis is thus only an approximation; it is not perfect.
5) More importantly, by analysing different algorithms, we can compare them to determine the best one for our purpose.

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

UNIT-1 (INTRODUCTION TO ALGORITHMS AND ANALYSIS)

SPACE AND TIME COMPLEXITY OF AN ALGORITHM

LECTURE NO:-2

In the world of computer science and digital products, the same thing occurs. To perform better, you need to write algorithms that are time efficient and use less memory.

An algorithm's complexity is a measure of the amount of data that it must process in order to be efficient. Domain and range of this function are generally expressed in natural units.

Time Complexity

Time complexity is defined in terms of how many times it takes to run a given algorithm, based on the length of the input. Time complexity is not a measurement of how much time it takes to execute a particular algorithm because such factors as programming language, operating system, and processing power are also considered. Time complexity is a type of computational complexity that describes the time required to execute an algorithm. The time complexity of an algorithm is the amount of time it takes for each statement to complete. As a result, it is highly dependent on the size of the processed data. It also aids in defining an algorithm's effectiveness and evaluating its performance.

Space Complexity

When an algorithm is run on a computer, it necessitates a certain amount of memory space. The amount of memory used by a program to execute it is represented by its space complexity. Because a program requires memory to store input data and temporal values while running, the space complexity is auxiliary and input space.

A good algorithm executes quickly and saves space in the process. You should find a happy medium of space and time (space and time complexity), but you can do with the average. Now, take a look at a simple algorithm for calculating the "mul" of two numbers.

UNIT-1 (INTRODUCTION TO ALGORITHMS AND ANALYSIS)

TYPES OF ASYMPTOTIC NOTATIONS AND ORDERS OF GROWTH

LECTURE NO:-3

Asymptotic Notations

Asymptotic Notations are programming languages that allow you to analyze an algorithm's running time by identifying its behavior as its input size grows. This is also referred to as an algorithm's growth rate. When the input size increases, does the algorithm become incredibly slow? Is it able to maintain its fast run time as the input size grows? You can answer these questions thanks to Asymptotic Notation.

You can't compare two algorithms head to head. It is heavily influenced by the tools and hardware you use for comparisons, such as the operating system, CPU model, processor generation, and so on. Even if you calculate time and space complexity for two algorithms running on the same system, the subtle changes in the system environment may affect their time and space complexity.

As a result, you compare space and time complexity using asymptotic analysis. It compares two algorithms based on changes in their performance as the input size is increased or decreased.

Asymptotic notations are classified into three types:

1. Big-Oh (O) notation
2. Big Omega (Ω) notation
3. Big Theta (Θ) notation

1. Big-Oh (O) Notation

Paul Bachmann invented the big-O notation in 1894. He inadvertently introduced this notation in his discussion of function approximation.

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

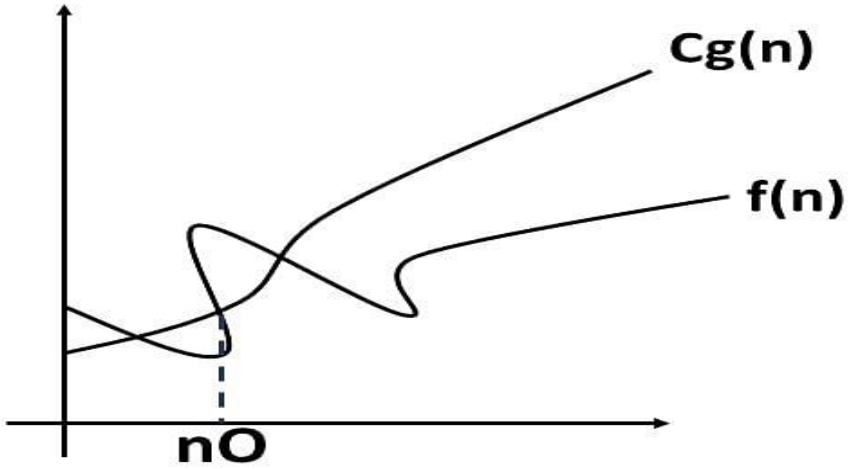
From the definition: O (g(n)) = {

f(n) : there exist positive constant c and n0 such that 0 <= f(n) <= c*g(n)

For all n >= n0

}

It is the most widely used notation for Asymptotic analysis. It specifies the upper bound of a function, i.e., the maximum time required by an algorithm or the worst-case time complexity. In other words, it returns the highest possible output value (big-O) for a given input.

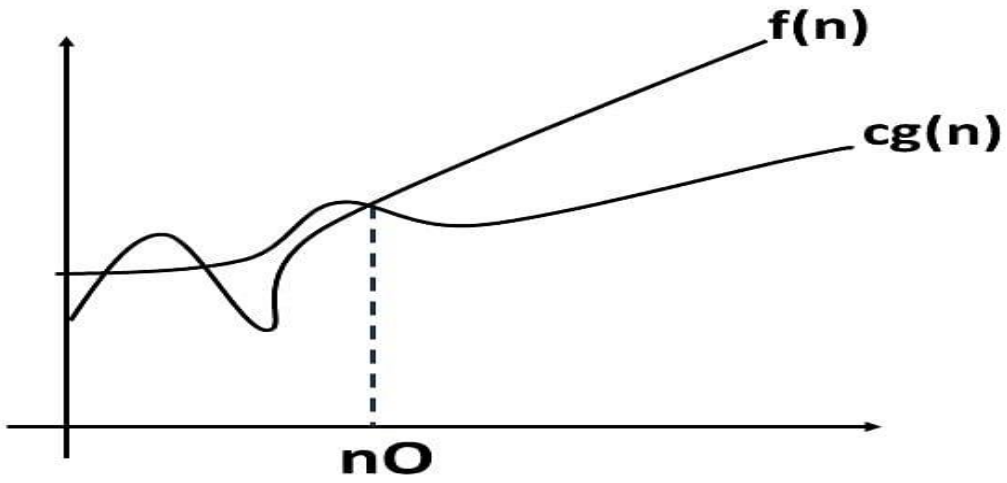


f(n) = O(g(n))

2. Big-Omega (Ω) notation

Big-Omega is an Asymptotic Notation for the best case or a floor growth rate for a given function. It gives you an asymptotic lower bound on the growth rate of an algorithm's runtime.

From the definition: The function f(n) is Ω (g(n)) if there exists a positive number c and N, such that f(n) >= cg(n) for all n >= N.



$$f(n) = \Omega(g(n))$$

3. Big-Theta (Θ) notation

Big theta defines a function's lower and upper bounds, i.e., it exists as both, most, and least boundaries for a given input value.

From the definition : f(n) is Θ(g(n)) if there exists positive numbers c1, c2 and N such that c1g(n) <= f(n) <= c2g(n) for all n >= N.

theta-notation-in-space-and-time-complexity.

Space Complexity

Space complexity refers to the total amount of memory space used by an algorithm/program, including the space of input values for execution. Calculate the space occupied by variables in an algorithm/program to determine space complexity.

However, people frequently confuse Space-complexity with auxiliary space. Auxiliary space is simply extra or temporary space, and it is not the same as space complexity. To put it another way,

Auxiliary space + space use by input values = Space Complexity

The best algorithm/program should have a low level of space complexity. The less space required, the faster it executes.

Time Complexity vs. Space Complexity

You now understand space and time complexity fundamentals and how to calculate it for an algorithm or program.

Time Complexity	Space Complexity
Calculates the time required	Estimates the space memory required
Time is counted for all statements	Memory space is counted for all variables, inputs, and outputs.
The size of the input data is the primary determinant.	Primarily determined by the auxiliary variable size
More crucial in terms of solution optimization	More essential in terms of solution optimization

UNIT-1 (INTRODUCTION TO ALGORITHMS AND ANALYSIS)

ALGORITHM EFFICIENCY – BEST CASE, WORST CASE, AVERAGE CASE

LECTURE NO:-4

Types of Algorithm Analysis:

1) Worst case
2) Best case
3) Average case

1. Worst Case Analysis (Mostly used)

In the worst-case analysis, we calculate the upper bound on the running time of an algorithm. We must know the case that causes a maximum number of operations to be executed. For Linear Search, the worst case happens when the element to be searched (x) is not present in the array. When x is not present, the search() function compares it with all the elements of arr[] one by one. Therefore, the worst-case time complexity of the linear search would be **O(n)**.

2. Best Case Analysis (Very Rarely used)

In the best-case analysis, we calculate the lower bound on the running time of an algorithm. We must know the case that causes a minimum number of operations to be executed. In the linear search problem, the best case occurs when x is present at the first location. The number of operations in the best case is constant (not dependent on n). So time complexity in the best case would be Ω(1)

3. Average Case Analysis (Rarely used)

In average case analysis, we take all possible inputs and calculate the computing time for all of the inputs. Sum all the calculated values and divide the sum by the total number of inputs. We must know (or predict) the distribution of cases. For the linear search problem, let us assume that all cases are uniformly distributed (including the case of x not being present in the array).

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

So we sum all the cases and divide the sum by (n+1). Following is the value of average-case time complexity.

MITRC

UNIT-1 (INTRODUCTION TO ALGORITHMS AND ANALYSIS)

ANALYSIS OF NON-RECURSIVE

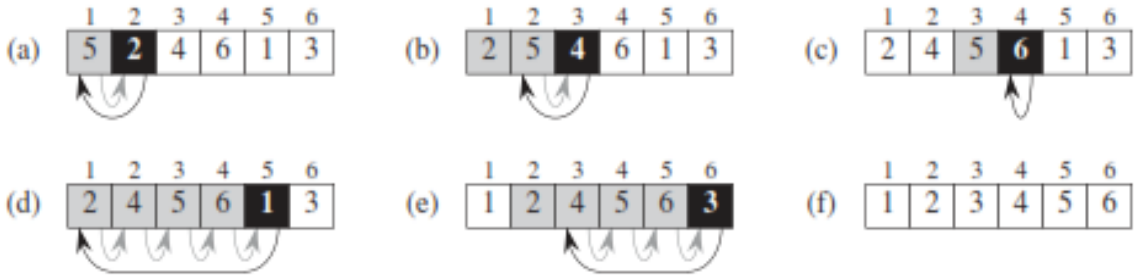
LECTURE NO:-5

Insertion sort

Input: A sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of the input sequence such that $a'_1 \le a'_2 \le \cdots \le a'_n$.

The numbers that we wish to sort are also known as the ***keys***. Although conceptually we are sorting a sequence, the input comes to us in the form of an array with n elements.



INSERTION-SORT(A)	*cost*	*times*
1 **for** $j = 2$ **to** $A.length$	c_1	n
2 $key = A[j]$	c_2	$n-1$
3 // Insert $A[j]$ into the sorted sequence $A[1..j-1]$.	0	$n-1$
4 $i = j-1$	c_4	$n-1$
5 **while** $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^{n} t_j$
6 $A[i+1] = A[i]$	c_6	$\sum_{j=2}^{n} (t_j - 1)$
7 $i = i-1$	c_7	$\sum_{j=2}^{n} (t_j - 1)$
8 $A[i+1] = key$	c_8	$n-1$

The running time of the algorithm is the sum of running times for each statement executed; a statement that takes c_i steps to execute and executes n times will contribute $c_i n$ to the total running time.[6] To compute $T(n)$, the running time of INSERTION-SORT on an input of n values, we sum the products of the *cost* and *times* columns, obtaining

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1) .$$

Even for inputs of a given size, an algorithm's running time may depend on *which* input of that size is given. For example, in INSERTION-SORT, the best case occurs if the array is already sorted. For each $j = 2, 3, \ldots, n$, we then find that $A[i] \leq key$ in line 5 when i has its initial value of $j - 1$. Thus $t_j = 1$ for $j = 2, 3, \ldots, n$, and the best-case running time is

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) . \end{aligned}$$

We can express this running time as $an + b$ for *constants* a and b that depend on the statement costs c_i; it is thus a ***linear function*** of n.

If the array is in reverse sorted order—that is, in decreasing order—the worst case results. We must compare each element $A[j]$ with each element in the entire sorted subarray $A[1 \ldots j-1]$, and so $t_j = j$ for $j = 2, 3, \ldots, n$. Noting that

$$\sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1$$

and

$$\sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2}$$

(see Appendix A for a review of how to solve these summations), we find that in the worst case, the running time of INSERTION-SORT is

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\ &\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n \\ &\quad - (c_2 + c_4 + c_5 + c_8) . \end{aligned}$$

We can express this worst-case running time as $an^2 + bn + c$ for constants a, b, and c that again depend on the statement costs c_i; it is thus a ***quadratic function*** of n.

Typically, as in insertion sort, the running time of an algorithm is fixed for a given input, although in later chapters we shall see some interesting "randomized" algorithms whose behavior can vary even for a fixed input.

UNIT-1 (INTRODUCTION TO ALGORITHMS AND ANALYSIS)

ANALYSIS OF RECURSIVE ALGORITHMS

LECTURE NO:-6

Recurrences

Recurrences go hand in hand with the divide-and-conquer paradigm, because they give us a natural way to characterize the running times of divide-and-conquer algorithms. A ***recurrence*** is an equation or inequality that describes a function in terms of its value on smaller inputs.

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 , \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 , \end{cases}$$

whose solution we claimed to be $T(n) = \Theta(n \lg n)$.

- In the ***substitution method***, we guess a bound and then use mathematical induction to prove our guess correct.
- The ***recursion-tree method*** converts the recurrence into a tree whose nodes represent the costs incurred at various levels of the recursion. We use techniques for bounding summations to solve the recurrence.
- The ***master method*** provides bounds for recurrences of the form

$$T(n) = aT(n/b) + f(n) ,$$

The substitution method for solving recurrences

Now that we have seen how recurrences characterize the running times of divide-and-conquer algorithms, we will learn how to solve recurrences. We start in this section with the "substitution" method.

The ***substitution method*** for solving recurrences comprises two steps:

1. Guess the form of the solution.
2. Use mathematical induction to find the constants and show that the solution works.

We substitute the guessed solution for the function when applying the inductive hypothesis to smaller values; hence the name "substitution method." This method is powerful, but we must be able to guess the form of the answer in order to apply it.

We can use the substitution method to establish either upper or lower bounds on a recurrence. As an example, let us determine an upper bound on the recurrence

$$T(n) = 2T(\lfloor n/2 \rfloor) + n ,$$

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

$T(n) = O(n \lg n)$. The substitution method requires us to prove that $T(n) \leq cn \lg n$ for an appropriate choice of the constant $c > 0$. We start by assuming that this bound holds for all positive $m < n$, in particular for $m = \lfloor n/2 \rfloor$, yielding $T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)$. Substituting into the recurrence yields

$$\begin{aligned} T(n) &\leq 2(c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \\ &\leq cn \lg(n/2) + n \\ &= cn \lg n - cn \lg 2 + n \\ &= cn \lg n - cn + n \\ &\leq cn \lg n , \end{aligned}$$

MITRC